

COMPACT

SW engineering redefined for the AI era

Context · **M**emory · **P**rompt

Audited · **C**o-developed · **T**eam-locked

A brief overview

Deep-dive walkthrough to follow

Why we built COMPACT

AI coding tools are powerful — but **solo by default**, and using them well is hard:

- 🧠 **Effective AI use takes deep understanding of AI internals** — context windows, prompt engineering, hallucination patterns. Most teammates don't have it; AI value concentrates among a few experts.
- 🔁 **Context resets every session.** AI re-reads files from scratch; yesterday's decisions are gone.
- ☹️ **Decisions evaporate.** *Why* did we pick library X? Buried in chat history that nobody else saw.
- 🌀 **Requirements / design / code drift apart.** Silent divergence accumulates across sessions.
- ⚡ **Parallel sessions collide.** Two devs editing the same status file, racing on the same decision log.
- 🤪 **AI hallucinates with confidence.** Invented APIs, fabricated rationale, plausible-sounding nonsense — at team scale, costly.

Without COMPACT, AI-assisted teams become AI-assisted solos — and AI value stays concentrated among the experts.

With COMPACT, the scaffold encodes the AI engineering — everyone benefits.

What COMPACT is

A **markdown-based scaffold** + **12 skills** that turn AI sessions into team-auditable, context-stable work.

- **Files are the truth.** State lives in `docs/compact/` and per-module `MODULE.md` files — not in chat. Every file is plain markdown, diff-reviewed in PR.
- **Phase discipline.** Requirements / Architecture / Development as explicit lenses the AI switches between deliberately.
- **Tool-neutral.** Works identically in **Claude Code** (Anthropic) and **Cline** (any LLM, including internal models). Same files, same skills.

The name is the contract: a *compact* is a binding agreement; *compact* is tightly-packed team lockstep.

Engineering DNA: Context + Prompt + Memory = *build, scale, and maintain large-scale SW solutions **without AI hallucinations**.*

© Context — progressive loading, no invention

The AI's **working memory is deliberately layered** so it always has what it needs and never has to fill gaps.

- **Tier 1 (always loaded at session start):** `PROJECT.md` · `STATUS.md` · `MAP.md` · active phase file
- **Tier 2 (on demand):** `MODULE.md` for what you're touching; `DECISIONS.md` entries when cited
- **Never pre-loaded:** other module files, code bodies, all decision entries — kept out of context until referenced
- **Strand-bound sessions** also load `STRAND.md` + recent journal tail
- **After auto-compaction:** re-invoke `/session-start` to reload Tier 1 from disk. Chat may forget; files don't.

*Everything the AI needs is already loaded. **It doesn't have to invent.***

P Prompt — EIP-tuned phase personas

Why EIP matters. Stock AI is sycophantic — agrees too easily, fills gaps with plausible-sounding inventions. **Emotional Intelligence Prompting** tunes the AI to behave like a *senior partner*: probes assumptions, surfaces trade-offs, names risks, pushes back.

Why a different persona per phase. Each phase needs a different cognitive mode. Without switching, modes bleed (AI solutions during requirements; debugs during architecture).

Phase	Persona behavior
Requirements	Analyst — exploratory; avoids solutioning , probes intent
Architecture	Architect — doc-first; flags over-engineering , one module at a time
Development	Engineer — contract-honoring; flags drift , small pieces

Stakeholders are part of the prompt — TPMs, QA, domain experts in `PROJECT.md` Contributors shape architecture from Day 1.

Phase prompts are the most load-bearing files in COMPACT.

Mechanism — see Anthropic's [Emotion concepts & function vectors](#) research.

M Memory — what persists between sessions

State lives in **files, not chat**. Memory is made deliberately at `/close-session`.

- **Memory is made at** `/close-session` — not mid-session. Decisions captured, STATUS updated by diff, MODULE.md edits audited. Never auto-writes.
- **DECISIONS.md is append-only** — entries are superseded, never edited. The history of *why* survives forever.
- **Strands extend memory to parallel work** — each in-flight item has its own journal + draft decisions, isolated until `/land-strand` promotes them.

Side benefit — memory layers become contracts.

`MODULE.md` = dev ↔ AI contract for module behavior.

`STATUS.md` / `DECISIONS.md` = team ↔ team coordination.

One scaffold, two collaboration modes.

Outcome — hallucination-free SW at scale

Combining **Context + Prompt + Memory** gives you:

- **Nothing to invent.** Context layer has everything; AI doesn't fill gaps with confident-sounding nonsense.
- **Trained to admit uncertainty.** EIP-tuned personas say *"let me re-read"* / *"can you confirm?"* instead of confabulating.
- **Deterministic where it matters.** `/regen-map` mechanically derives module structure (AST, never AI); `/close-session` never invents rationale; `/drift-check` shows `file:line` evidence across R/D/I; `/doctor` audits the scaffold.
- **Propose-don't-write throughout.** Every state change is a user-approved diff. No silent writes.

The AI cannot hallucinate what the protocol forces it to re-check.

Same protocol scales from 1-person prototypes to multi-team systems — files are the constant.

Twelve skills, three groups

Project Primitives — project-level state

`/project-init` · `/switch-phase` · `/regen-map` · `/drift-check`

Session Primitives — what every session does

`/session-start` · `/close-session` · `/doctor`

Parallel Work Primitives — strands for multi-track work

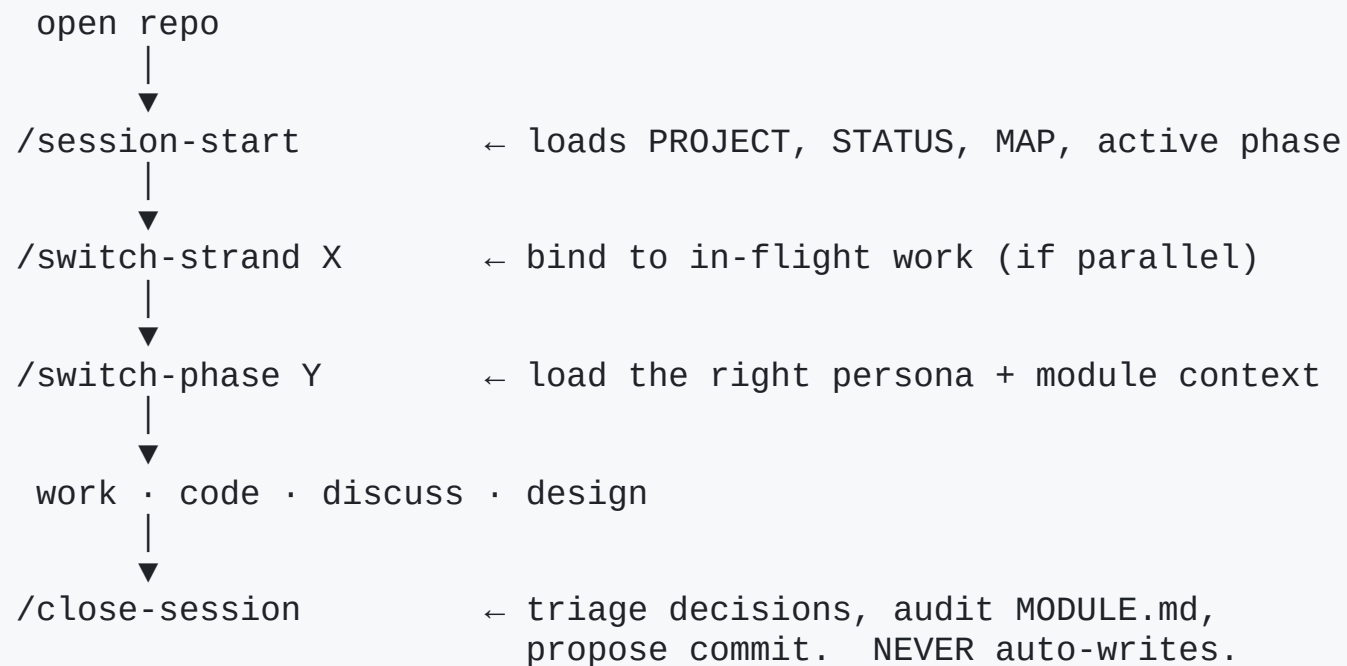
`/start-strand` · `/switch-strand` · `/list-strands` · `/land-strand` · `/adopt-strands`

- **C P M** — engineering DNA
- 12 skills · 3 groups
- 6 use-case flows
- Context tiers per phase

You'll get a copy.

CORE PRINCIPLE - Files are the truth. Chat is scratch space. Memory gets made at /close-session — never mid-session.

A typical session



Memory is made at `/close-session` . Mid-session is scratch space.

Parallel work via strands

When multiple devs or multiple features run in parallel, **strands** prevent collision.

- Each strand = one folder under `docs/compact/strands/<name>/`
- Contains: `STRAND.md` (status + assignees + target modules), `journal.md` (per-session log), `decisions-draft.md` (draft ADRs)
- **Per-clone binding** (`.compact/current-strand` , gitignored) — teammates can each be in a different strand on the same repo
- `/land-strand` promotes draft decisions to canonical `DECISIONS.md` with sequential `D-XXX` , archives the folder

Multiple in-flight items no longer fight over `STATUS.md` or `DECISIONS.md` numbering.

How a team adopts it

Greenfield project

`/project-init` → 7-topic interview → scaffolds `docs/compact/` + customized phase prompts. Starts strand-ready.

Existing project

`/project-init --retrofit` → codebase scan + `MODULE.md` skeletons + polyglot-aware `structure-conventions.md`
(optional) `/adopt-strands` → seed strands from current `STATUS.md` in-flight items

Per session, in any project

Install the skills once into `.claude/skills/`. Both Claude Code and Cline pick them up automatically — no separate config.

Cost to adopt: one `project-init` invocation + ~5 min team onboarding.

COMPACT = SW engineering redefined for the AI era

Era	What changed how we ship
1990s	Version control — distributed dev becomes possible
2000s	Code review + tests — quality scales
2010s	CI/CD — deployment becomes continuous
Now	AI-partnered teams need their own protocol

- **Level playing field.** Same scaffolding for everyone — junior/senior, AI-savvy or not. Prompt engineering is built in; no AI background needed.
- **Common pitfalls designed out.** Hallucination, lost context, opaque decisions, drift — guarded by skills, not heroics.
- **One scaffold, two modes.** Memory files are both **human ↔ AI contracts** and **human ↔ human coordination**.

Code review, VCS, CI/CD — each was once "the new thing."
COMPACT is the protocol layer for AI-partnered teams.

What's next

- **Deep-dive session** — ~1 hour walkthrough with a real dry-run (*date TBD*)
- **Pilot candidate** — pick one project (greenfield or retrofit) and run through a real cycle
- **Source** — `github.com/kurnoolion/utils` → `skills/compact/`
- **One-page reference** — `COMPACT_Cheatsheet.png` in the source repo
- **Full deck** — `COMPACT_Overview.md` (Marp deck for the deep-dive)

Questions?